

Microsoft Teams 企業級通知服務系統

Request for Quotation (RFQ)

專案代碼: RFQ-2025-TEAMS-NOTIFY-001

發布日期: 2025年10月23日

委託單位: [公司名稱]

聯絡窗口: 採購部門 / IT 部門

簡報議程

1. 專案背景與目標 (5 分鐘)
2. 系統架構概述 (5 分鐘)
3. 核心功能模組 (10 分鐘)
4. 技術規格要點 (10 分鐘)
5. 專案時程與里程碑 (5 分鐘)
6. 預算與資源需求 (5 分鐘)
7. 廠商資格與評選標準 (5 分鐘)
8. 風險與挑戰 (5 分鐘)
9. 下一步行動 (5 分鐘)

總時長: 約 55 分鐘 + Q&A 15 分鐘

1 專案背景與目標

專案背景

現況問題

- 公司內部有多個應用系統需要發送 Teams 通知
- 目前各系統各自整合 Teams Bot，管理複雜
- 缺乏統一的通知管理與使用量追蹤
- 需要企業級的安全性與可靠性保障

解決方案

- ✓ 建立統一的 Teams 通知中台
- ✓ 提供 RESTful API
- ✓ 支援多目標通知
- ✓ 實現計費追蹤與權限管理
- ✓ 確保高可用性與高性能

預期效益

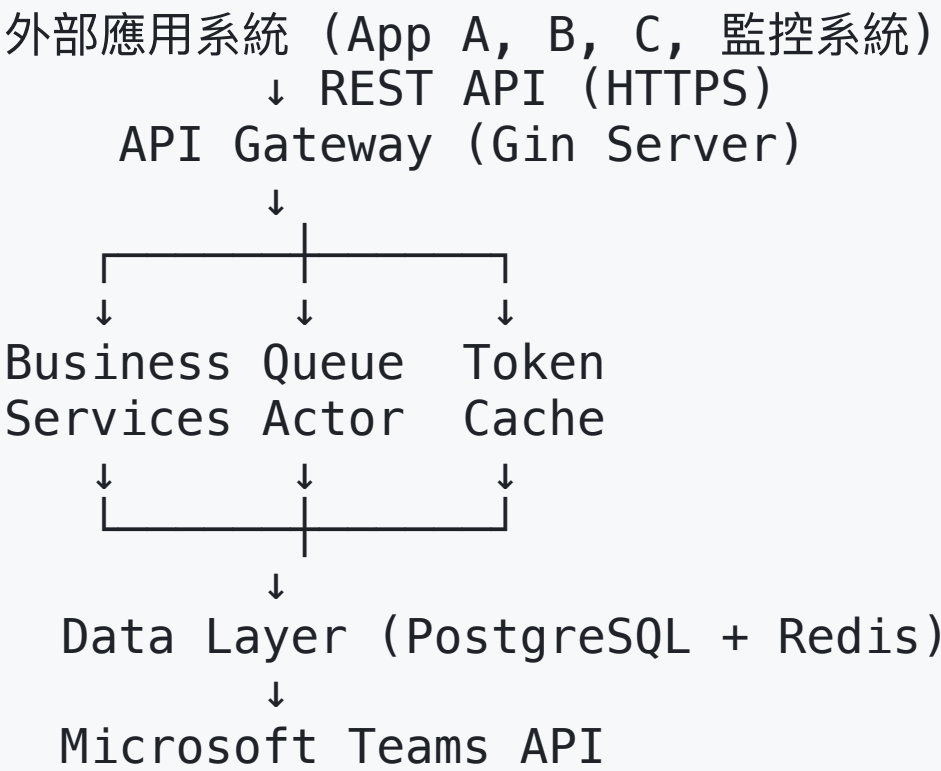
效益類別	說明	預期改善
 成本節約	避免重複開發、統一維護	降低維護成本 30%
 效率提升	新系統整合時間縮短	從 2 週降至 2 天
 風險降低	統一認證、審計、權限管理	提升安全性與合規性

關鍵數字

- 系統可用性: $\geq 99.5\%$
- API 響應時間 (P95): $< 500\text{ms}$
- 併發支援: $\geq 100 \text{ req/s}$

2 系統架構概述

技術架構



關鍵技術特性

特性	技術實作	效益
Two-loop Enqueue	Producer 快速寫 DB + Worker 批次入隊	API 延遲 < 5ms
Actor Pool	Goroutine Pool 管理並發	可控並發、防止資源耗盡
Token Cache	Redis 快取 50min TTL	減少 API 調用 95%+
Circuit Breaker	熔斷器保護	快速失敗、防止級聯故障

技術棧選型

類別	技術	版本	選擇理由
程式語言	Golang	1.21+	高性能、併發優秀、記憶體效率高
Web 框架	Gin	1.9+	輕量、快速、生態成熟
資料庫	PostgreSQL	14+	關聯查詢強、JSONB 支援
快取/佇列	Redis	7+	高性能、持久化、支援多種資料結構
容器化	Docker	20+	標準化部署、易於擴展

為什麼選擇 **Golang** ？ 原生高併發、部署簡單、記憶體效率比 Java/Node.js 低 50%+

3 核心功能模組

13 個功能模組

P0 必須實作

- M-001: 公司與用戶管理
- M-002: 專案管理
- M-003: Teams Bot 管理
- M-004: 目的地管理
- M-005: 通知發送服務
- M-006: 佇列與重試系統

P1 建議實作

- M-007: Token 管理
- M-008: 計費系統
- M-009: 監控與日誌
- M-010: 外部 API

P2 可選實作

- M-011: 檔案管理
- M-012: 審計日誌
- M-013: 報表分析

核心功能 1: 預先註冊機制

Notification Key 映射流程

```
外部系統 → POST /api/v1/notify
           { "notifyKey": "project-alpha",
             "targets": ["all"] }
           ↓
系統處理 → 1. 驗證 notifyKey
           2. 查詢預先註冊的目的地
           3. 展開 targets (all → 所有目的地)
           4. 批次發送通知
```

優點

- ✓ 安全可控 (預先註冊)
- ✓ 集中管理 (IT 統一管理)
- ✓ 權限隔離 (專案隔離)
- ✓ 審計追蹤 (完整日誌)

核心功能 2: Two-loop Enqueue 設計

三階段處理

- 1 Producer Loop (API)
接收請求 → 快速寫入 DB → 回傳 202 Accepted
🕒 延遲: 1-5ms
- 2 Enqueue Worker (背景)
掃描 pending → 推入 Redis → 按優先級排序
- 3 Consumer Loop (Actor Pool)
BLPOP → Actor 處理 → 重試 + 熔斷器 → sent/failed

關鍵優勢: 低延遲 + 高吞吐 + 可靠性 + 可觀測性

核心功能 3: 智能重試機制

指數退避策略

重試次數	等待時間	累計時間
第 1 次失敗	1 秒	1 秒
第 2 次失敗	2 秒	3 秒
第 3 次失敗	4 秒	7 秒
第 4 次失敗	8 秒	15 秒
第 5 次失敗	16 秒	31 秒

特殊處理: 429 Rate Limit → 讀取 Retry-After header

熔斷器保護

Closed → (5 次失敗) → Open (30 秒) → Half-Open → Closed/Open

4 技術規格要點

 API 規模總覽

總計 100+ API 端點

API 類別	端點數	優先級
系統端點（健康檢查、指標）	2	P0/P1
外部 API（Notify、Provision）	8	P0/P1
公司管理	6	P0
用戶管理	8	P0
專案管理	7	P0
Bot 管理	8	P0
目的地管理	9	P0
通知管理	9	P0
其他（計費、檔案、監控、佇列）	25+	P1/P2

資料庫設計

12 個核心資料表

- **companies** (公司) → users, projects
- **projects** (專案) → destinations, notifications, usage_records, project_billing
- **teams_bots** (Bot) → destinations
- **destinations** (目的地) ← **JSONB targets** 欄位
- **notifications** (通知) → notification_destinations
- **notification_destinations** (分派) ← **按月分區**
- **billing_plans** (計費方案) → project_billing
- **files** (檔案)、**audit_logs** (審計日誌)

關鍵設計: JSONB 欄位、分區策略、GIN/B-tree 索引、彙總檢視

非功能需求

性能指標

指標	目標值	測試方法
API 響應時間 (P95)	< 500ms	Load Testing
API 響應時間 (P99)	< 1000ms	Load Testing
通知發送成功率	≥ 99%	Production Monitoring
併發請求	≥ 100 req/s	Load Testing
佇列處理能力	≥ 1000 notif/min	Performance Testing

可用性指標

- 系統可用性: ≥ 99.5% (每月停機 < 3.6 小時)
- MTTR: < 30 分鐘

 安全性規格

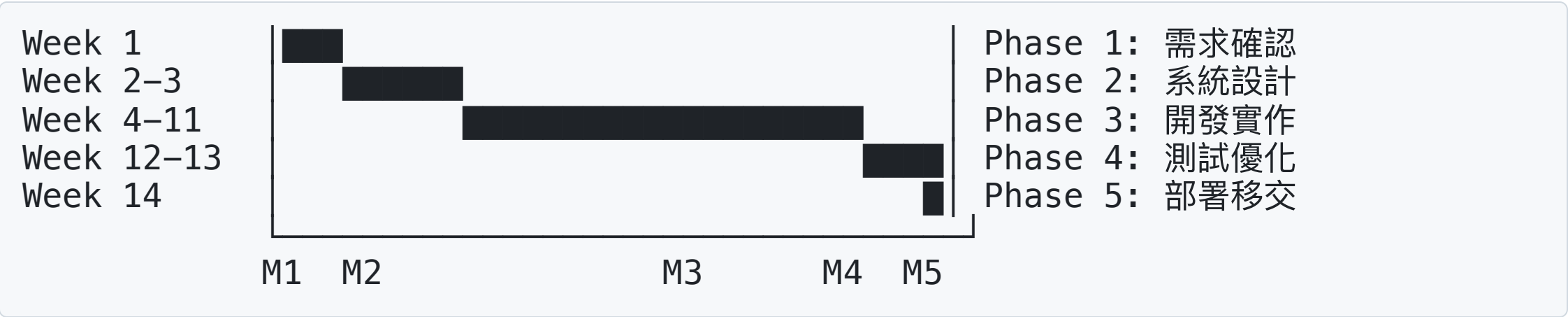
認證與授權

機制	實作
認證	JWT Token + API Key + OAuth2
授權	RBAC (Admin/Manager/User)
加密	HTTPS/TLS 1.2+, AES-256
密碼	bcrypt, 8+ 字元
防護	SQL 防注入、Rate Limiting
審計	完整日誌記錄

5 專案時程與里程碑

JUL 17

14 週時程規劃



5 個里程碑

- M1 (Week 1): 需求規格書簽署
- M2 (Week 3): 設計審查通過
- M3 (Week 7): Alpha 版本交付
- M4 (Week 11): Beta 版本交付
- M5 (Week 14): 正式驗收通過

交付物清單

軟體交付物（8 項）

1.  原始碼（Go） - Git Repository
2.  Database Schema - SQL 檔案
3.  API 文件 - OpenAPI 3.0
4.  Docker 映像 - 可執行映像檔
5.  部署配置 - Kubernetes YAML
6.  單元測試 - 覆蓋率 $\geq 70\%$
7.  整合測試 - Shell/Go 測試腳本
8.  操作手冊 - Markdown 文件

技術移交

6 預算與資源需求

 預算估算

項目	預估金額 (萬元)	佔比
開發費用	80-120	60%
文件撰寫	15-20	12%
測試品保	20-30	15%
技術移交	8-12	6%
專案管理	10-15	7%
總預算	133-197	100%

付款條件（建議）

- 簽約時: 30%
- Alpha 版本: 30%
- Beta 版本: 20%
- 驗收通過: 20%

人力配置要求

最低團隊配置（4-5 人）

角色	人數	要求
專案經理	1	PMP 或同等證照
資深後端工程師	1	Golang 3 年以上
後端工程師	1-2	Golang 2 年以上
QA 工程師	1	測試經驗 2 年以上
技術文件	1	可兼任

技術能力要求（至少 3 項）

- ☒ Golang 後端開發
- ☒ Teams Bot/Graph API 整合
- ☒ RESTful API 設計
- ☒ PostgreSQL/Redis 經驗
- ☒ Docker/Kubernetes 部署
- ☒ 企業級系統開發

7 廠商資格與評選標準

廠商資格要求

基本資格

- 合法登記之公司行號
- 具備軟體開發能力之技術團隊
- 公司成立滿 **2 年**以上
- 無重大違約或訴訟紀錄

專案經驗

- 提供近 **3 年**內類似專案案例（至少 2 個）
- 專案規模、技術棧、專案成果說明
- 客戶推薦函或專案證明文件

 評選標準



必要淘汰條件

-  不符合基本資格
-  報價文件不完整
-  逾期提交報價
-  提供不實資訊

8 風險與挑戰

 風險矩陣

風險類型	風險項目	影響	機率	應對措施
技術	Teams API 變更	高	中	版本鎖定、定期更新
技術	高併發性能問題	高	中	負載測試、優化調校
技術	整合複雜度高	中	高	技術驗證 POC
業務	需求變更頻繁	中	中	變更管理流程
業務	廠商資源不足	高	低	嚴格資格審查
業務	時程延遲	中	中	里程碑管控

專案成功關鍵因素

技術面

1. 廠商技術能力強
2. 架構設計合理
3. 測試覆蓋完整
4. 文件齊全

管理面

1. 需求明確
2. 時程管控
3. 溝通順暢
4. 變更管理

商業面

- 合理預算、分階段付款、保固支援、技術移交

9 下一步行動

JUL 17

時間軸

現在	2025/10/23	● RFQ 發布
13 天後	2025/11/05	● 廠商提問截止
16 天後	2025/11/08	● 統一答覆發布
23 天後	2025/11/15	● 報價截止 ⚠
28-30 天後	2025/11/20	● 內部評選
33-35 天後	2025/11/25	● 廠商簡報
38 天後	2025/11/30	● 決標公告 🎉
48 天後	2025/12/10	● 合約簽訂 ✍
53 天後	2025/12/15	● 專案啟動 🚀

立即行動項目

採購部門

- ☐ 發布 RFQ 到廠商平台
- ☐ 建立廠商提問管道
- ☐ 準備評選委員會名單

IT 部門

- ☐ 回答廠商技術問題
- ☐ 準備技術評選標準
- ☐ 協調測試環境

管理層

- ☐ 核准預算範圍

- ☐ 指定評選委員

? Q&A

常見問題

Q1: 為什麼選擇外包？

✅ 專業度高、時間快（14 週 vs 6 個月）、避免招聘成本、風險可控

Q2: 預算是否合理？

市場行情 100-200 萬、本案 15K LOC + 100+ API + 14 週 → 合理

Q3: 如何確保按時交付？

分階段驗收、分階段付款、延遲罰則、定期檢查

Q4: 智財權如何處理？

所有程式碼、文件智財權歸本公司，合約明確規定



總結

專案價值

- ✓ 統一管理 - 降低維護成本 30%
- ✓ 快速整合 - 2 天即可整合 Teams
- ✓ 企業級 - 高可用 99.5%、高性能、安全可靠
- ✓ 完整追蹤 - 計費、審計、監控一應俱全

關鍵數字

- API 端點: 100+
- 開發工期: 14 週
- 預估預算: 133-197 萬
- 團隊規模: 4-5 人
- 測試覆蓋: $\geq 70\%$
- 系統可用性: $\geq 99.5\%$

下一步

1. **11/05** - 廠商提問截止
2. **11/15** - 報價截止 ⚠️
3. **11/30** - 決標公告
4. **12/15** - 專案啟動

謝謝聆聽！

Questions?

聯絡資訊

 採購相關: procurement@company.com

 技術相關: it-rfq@company.com

RFQ 文件下載: [連結]

提案截止日期: 2025/11/15 17:00

祝各位廠商提案順利！

期待收到您的優秀提案 🎉